

REPORTE • UNIDAD 1

Paradigmas de programación: historia, uso actual y su aplicación en un videojuego

Del programa de cajas y pelotas que construimos en clase a un Geometry Dash en C#, y su comparación con una versión procedural generada con IA.

Materia	Paradigmas de Programación
Alumno	José Franyutti Ricardez
Equipo del código	José Franyutti Ricardez y Eduardo Orduño Moreno
Lenguaje	C# con Raylib-cs
Fecha	5 de octubre de 2026



1. Introducción

Durante la Unidad 1 el profesor construyó, clase por clase, un programa en C# con la biblioteca Raylib: una ventana donde cajas y pelotas se mueven, rebotan contra los bordes y cambian de color al chocar. Ese programa sirvió para ver en código real los paradigmas imperativo, estructurado, orientado a objetos y dirigido por eventos. Como práctica, mi compañero Eduardo Orduño Moreno y yo lo extendimos hasta convertirlo en un juego tipo *Geometry Dash*: un cubo que salta obstáculos, lleva puntaje y muestra una pantalla de derrota.

Este reporte tiene tres partes. Primero investigo los paradigmas de programación que se usan actualmente y los ubico en una línea del tiempo. Después explico cómo se aplican en nuestro juego. Al final presento una versión procedural del mismo juego, generada con IA como pide la actividad, y comento las diferencias entre las dos formas de programarlo.

2. ¿Qué es un paradigma de programación?

Un paradigma de programación es una forma de pensar y organizar la solución de un problema en código. No es un lenguaje: es el estilo con el que se usa. Define qué elementos forman el programa (instrucciones, funciones, objetos, reglas) y cómo se relacionan. Un mismo lenguaje puede permitir varios paradigmas; C#, por ejemplo, permite escribir código orientado a objetos, procedural y funcional.

Los paradigmas suelen agruparse en dos grandes familias:

- **Imperativos:** el programa dice *cómo* hacer las cosas, paso a paso, cambiando el estado de la memoria. Aquí están el imperativo puro, el procedural, el estructurado y el orientado a objetos.
- **Declarativos:** el programa dice *qué* resultado se quiere y deja que el lenguaje decida cómo obtenerlo. Aquí están el funcional, el lógico y lenguajes como SQL.

El paradigma dirigido por eventos es transversal: describe cómo fluye el programa (reaccionando a teclas, clics o temporizadores) y se combina con cualquiera de los anteriores.

3. Paradigmas utilizados actualmente

■ Imperativo

Es el paradigma más cercano a cómo funciona la computadora. El programa es una secuencia de órdenes que leen y modifican variables; el resultado depende del orden en que se ejecutan.

- Uso de variables y asignaciones que cambian el estado.
- Ejecución secuencial, con ciclos y condiciones para controlar el flujo.

- Es la base sobre la que se construyen el paradigma procedural y el orientado a objetos.

Lenguajes representativos: ensamblador, C, Fortran y, en general, la parte de bajo nivel de casi todos los lenguajes.

```
puntos++;
velocidad.Y += 0.8f;      // gravedad
posición.Y += velocidad.Y;
```

Ejemplo 1. Instrucciones imperativas dentro del ciclo de nuestro juego.

■ Procedural / estructurado

Organiza las instrucciones imperativas en procedimientos o funciones reutilizables que reciben datos y los transforman. La programación estructurada, impulsada por Edsger Dijkstra en 1968, agrega la regla de construir todo con tres estructuras: secuencia, selección (if) e iteración (ciclos), evitando los saltos con goto.

- El programa se divide en funciones con una tarea clara.
- Los datos y las funciones están separados; los datos suelen pasarse como parámetros o vivir en variables globales.
- El flujo se lee de arriba hacia abajo, lo que lo hace fácil de seguir.

Lenguajes representativos: C, Pascal, COBOL, Go y el estilo de scripts en Python.

■ Orientado a objetos

Modela el problema como un conjunto de objetos que juntan datos (atributos) y comportamiento (métodos). Nació con Simula 67 y se popularizó con Smalltalk, C++, Java y C#. Es el paradigma central del programa de clase.

- **Abstracción:** se definen clases con lo esencial; las clases abstractas declaran métodos sin implementarlos.
- **Encapsulamiento:** cada objeto guarda su propio estado y lo modifica con sus métodos.
- **Herencia:** una clase hija reutiliza y especializa a la clase padre.
- **Polimorfismo:** el mismo mensaje, como *Update()*, produce comportamientos distintos según el objeto.

Lenguajes representativos: C#, Java, C++, Python, Kotlin y Swift.

```
abstract class Objeto : IComparable<Objeto>
{
    public abstract void Draw();
    public abstract void Update(int screenWidth, int screenHeight);
}
class Caja : Objeto { ... }
class Jugador : Caja { public override void Update(...) { ... } }
```

Ejemplo 2. Abstracción, herencia y sobrescritura en nuestro código.

■ Dirigido por eventos

El orden de ejecución no lo decide una lista fija de instrucciones sino los eventos externos: una tecla, un clic, un mensaje de red o un temporizador. El programa registra qué hacer ante cada evento y espera.

- Existe un ciclo principal que revisa eventos y llama al código que responde.
- Es la base de las interfaces gráficas, los videojuegos y las páginas web.
- En Raylib se consulta el estado del teclado en cada cuadro del juego.

Lenguajes representativos: JavaScript, C#, Visual Basic y frameworks de interfaces gráficas.

```
if (enSuelo && Raylib.IsKeyDown(KeyboardKey.Space))
{
    velocidad.Y = -15f;
    enSuelo = false;
}
```

Ejemplo 3. El salto responde al evento de la barra espaciadora.

■ Funcional

Construye programas componiendo funciones puras, que siempre devuelven lo mismo para la misma entrada y no modifican datos externos. Viene del cálculo lambda y de LISP (1958).

- Datos inmutables y ausencia de efectos secundarios.
- Funciones de orden superior: funciones que reciben o devuelven otras funciones.
- Se adapta bien a la programación concurrente y al procesamiento de datos.

Lenguajes representativos: Haskell, Lisp, F#, Elixir y el uso de lambdas y LINQ en C#.

```
var grandes = objetos.Where(o => o.GetArea() > 2000)
                    .OrderBy(o => o.GetArea());
```

Ejemplo 4. Estilo funcional con LINQ dentro de C#.

■ Lógico

El programador escribe hechos y reglas, y el sistema deduce respuestas mediante inferencia lógica. Su lenguaje principal es Prolog (1972) y se usó mucho en la inteligencia artificial simbólica.

- Se describe el conocimiento, no el algoritmo.
- El motor de inferencia busca automáticamente las soluciones.
- Útil en sistemas expertos, procesamiento de lenguaje y verificación.

Lenguajes representativos: Prolog, Datalog.

■ Declarativo

Se describe el resultado deseado sin indicar los pasos. El ejemplo más usado es SQL: se pide qué datos se quieren y el gestor de base de datos decide cómo buscarlos. También es la idea detrás de HTML y de bibliotecas de interfaces como React.

- Mayor nivel de abstracción y código más corto.
- El sistema optimiza la forma de ejecutar la petición.

Lenguajes representativos: SQL, HTML, CSS, React.

■ Multiparadigma

La mayoría de los lenguajes modernos no se limitan a un paradigma. Permiten mezclar estilos según convenga: clases para modelar el dominio, funciones puras para transformar datos y eventos para la interfaz.

- C# es orientado a objetos, pero incluye lambdas, LINQ y tipos de registro inmutables.
- Python permite escribir scripts procedurales, clases y código funcional.

Lenguajes representativos: C#, Python, C++, JavaScript, Kotlin, Rust.

3.1 ¿Qué paradigmas dominan hoy?

Para ver qué se usa actualmente consulté el índice TIOBE de septiembre de 2026, que mide la popularidad de los lenguajes según búsquedas en internet. La tabla relaciona los diez primeros con sus paradigmas:

#	Lenguaje	Rating	Paradigmas principales
1	Python	17.76 %	Multiparadigma: procedural, OO, funcional
2	C	10.28 %	Imperativo, procedural, estructurado
3	C++	8.67 %	Multiparadigma: procedural, OO, genérico
4	Java	7.54 %	Orientado a objetos, con elementos funcionales
5	C#	4.22 %	Orientado a objetos, funcional, eventos
6	JavaScript	2.76 %	Eventos, funcional, OO basado en prototipos
7	Visual Basic	2.55 %	Orientado a objetos, eventos
8	SQL	2.16 %	Declarativo
9	R	1.69 %	Funcional, procedural (estadística)
10	Rust	1.34 %	Multiparadigma: imperativo, funcional

Tabla 1. Diez lenguajes más populares según el índice TIOBE de septiembre de 2026 y sus paradigmas.

De la tabla saco tres conclusiones. Primero, el paradigma orientado a objetos sigue siendo el más extendido: aparece en siete de los diez lenguajes. Segundo, los lenguajes imperativos y procedurales como C siguen en lo más alto porque se usan en sistemas

operativos, microcontroladores y bibliotecas como Raylib. Tercero, la tendencia es multiparadigma: Python ocupa el primer lugar en buena parte por la inteligencia artificial y la ciencia de datos, y permite combinar varios estilos en el mismo programa.

4. Línea del tiempo de los paradigmas

La figura muestra cuándo apareció cada paradigma y los hitos principales de cada uno. Cada barra empieza en el año en que surgió el paradigma y llega hasta hoy, porque todos se siguen usando.

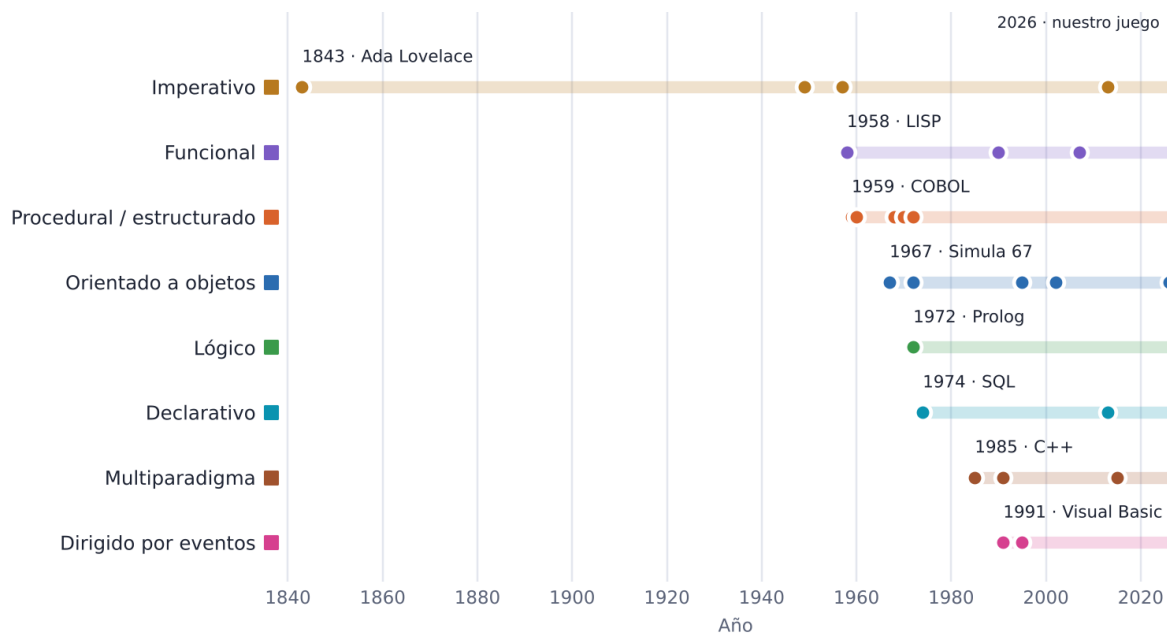


Figura 1. Aparición y vigencia de los paradigmas de programación, de 1843 a 2026.

Año	Hito	Paradigma	Descripción
1843	Notas de Ada Lovelace	Imperativo	Primer algoritmo pensado para una máquina: la máquina analítica de Babbage.
1949	Lenguaje ensamblador	Imperativo	Mnemónicos en lugar de números binarios.
1957	FORTRAN	Imperativo	Primer lenguaje de alto nivel de uso amplio (John Backus, IBM).
1958	LISP	Funcional	Funciones, listas y recursión (John McCarthy). Base de la IA simbólica.
1959	COBOL	Procedural / estructurado	Lenguaje para negocios organizado en procedimientos.
1960	ALGOL 60	Procedural / estructurado	Bloques, alcance de variables y recursión.
1967	Simula 67	Orientado a objetos	Primeras clases, objetos y herencia (Dahl y Nygaard).
1968	Programación estructurada	Procedural / estructurado	Dijkstra propone eliminar el goto.

Año	Hito	Paradigma	Descripción
1970	Pascal	■ Procedural / estructurado	Lenguaje para enseñar programación estructurada (Wirth).
1972	C	■ Procedural / estructurado	Funciones, structs y punteros (Ritchie, Bell Labs).
1972	Prolog	■ Lógico	Hechos y reglas con inferencia lógica.
1972	Smalltalk	■ Orientado a objetos	Todo es un objeto; objetos que se envían mensajes (Kay, Xerox PARC).
1974	SQL	■ Declarativo	Consultas que describen qué datos se quieren (IBM).
1985	C++	■ Multiparadigma	C con clases: procedural y orientado a objetos (Stroustrup).
1990	Haskell	■ Funcional	Lenguaje funcional puro.
1991	Python	■ Multiparadigma	Procedural, OO y funcional (Van Rossum).
1991	Visual Basic	■ Dirigido por eventos	Popularizó programar respondiendo a eventos de una interfaz.
1995	Java	■ Orientado a objetos	Orientado a objetos y portable gracias a su máquina virtual.
1995	JavaScript	■ Dirigido por eventos	Eventos en las páginas web (Eich, Netscape).
2002	C# y .NET	■ Orientado a objetos	Lenguaje de nuestro juego (Microsoft, Hejlsberg).
2007	C# 3.0 y LINQ	■ Funcional	Lambdas y consultas funcionales dentro de un lenguaje OO.
2013	raylib	■ Imperativo	Biblioteca de videojuegos con un ciclo simple (Santamaria).
2013	React	■ Declarativo	Interfaces descritas a partir del estado.
2015	Rust 1.0	■ Multiparadigma	Seguridad de memoria sin recolector de basura.
2026	Nuestro Geometry Dash	■ Orientado a objetos	Extensión OO del programa de clase en C# con Raylib-cs.

Tabla 2. Hitos principales de la historia de los paradigmas.

También preparamos una versión interactiva de esta línea del tiempo, llamada **Paradigm Dash**: un juego web en el que el cubo recorre los mismos hitos y cada portal muestra uno de ellos. El archivo está en la carpeta docs/ del repositorio.

5. Del programa de clase a nuestro Geometry Dash

5.1 El programa base

El programa de clase define una clase abstracta `Objeto` que implementa `IComparable<Objeto>` para ordenar los objetos por área. De ella heredan `Caja` y `Pelota`, y de `Caja` hereda `Raqueta`, que se mueve con las flechas del teclado. En `Main` se crea una `List<Objeto>` y el ciclo del juego actualiza cada objeto, revisa las colisiones entre todos los pares y los dibuja. Al detectar un choque, ambos objetos se pintan de rojo.

5.2 Nuestra extensión

Mantuvimos la jerarquía y la idea de la lista polimórfica, y agregamos dos clases nuevas que heredan de `Caja`:

- **Jugador:** un cubo de 50×50 que sobrescribe `Update()` para aplicar gravedad (0.8 por cuadro), saltar con la barra espaciadora solo cuando está en el suelo y detenerse al tocarlo.
- **Obstaculo:** avanza hacia la izquierda a 6 píxeles por cuadro y, cuando sale de la pantalla, reaparece 1800 píxeles a la derecha. Así el nivel nunca termina.

Además agregamos el estado del juego (puntos y muerto), la pantalla de **¡PERDISTE!** con el puntaje, el reinicio con la barra espaciadora y la función `CrearObstaculos()`. Quitamos `Pelota` y `Raqueta` y simplificamos `Caja.Update()`, porque el juego no necesita rebotes contra las paredes.

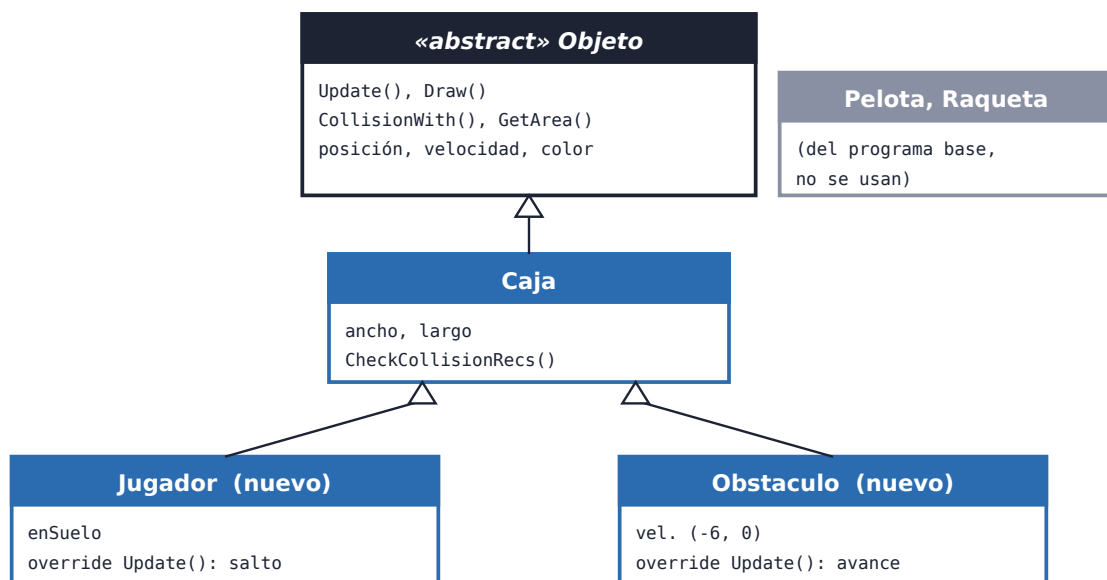


Figura 2. Diagrama de clases de nuestro juego. Las flechas indican herencia.

5.3 Paradigmas presentes en el juego

Paradigma	Dónde aparece en nuestro código
■ Orientado a objetos	Clase abstracta Objeto; herencia Caja → Jugador / Obstaculo; override de Update(); lista polimórfica List<Objeto>; interfaz IComparable.
■ Imperativo	El ciclo while (!WindowShouldClose()) que actualiza, revisa choques y dibuja 60 veces por segundo; las asignaciones que cambian posición, velocidad y puntaje.
■ Procedural / estructurado	Funciones como CrearObstaculos(objetos, sueloY) y el uso de if y ciclos sin goto.
■ Dirigido por eventos	IsKeyDown(Space) para saltar e IsKeyPressed(Space) para reiniciar después de perder.

Tabla 3. Relación entre los paradigmas de la Unidad 1 y el código del juego.

El polimorfismo es lo que más se nota: el ciclo principal recorre la lista y llama a `o.Update()` sin saber si cada elemento es un obstáculo o cualquier otra subclase. Para agregar un nuevo tipo de obstáculo, por ejemplo uno que se mueva arriba y abajo, bastaría con crear otra clase hija de Caja y agregarla a la lista, sin tocar el ciclo principal.

6. Versión procedural generada con IA

Como pide la actividad, usé una herramienta de inteligencia artificial (Claude, de Anthropic) para reescribir nuestro juego en estilo procedural. Le di el código orientado a objetos y le pedí que conservara exactamente el mismo comportamiento (gravedad, salto, obstáculos, puntaje y pantalla de derrota), pero sin clases con herencia, sin métodos virtuales y sin polimorfismo. Revisé el resultado contra el original para comprobar que cada función hace lo mismo que el método correspondiente.

La versión procedural se organiza así:

- Un struct `Bloque` que solo guarda datos: posición, velocidad, tamaño y color, como un struct de C.
- Variables globales para el estado: el jugador, un arreglo de tres obstáculos, el puntaje y si el jugador perdió.
- Funciones independientes: `ReiniciarJuego()`, `ActualizarJugador()`, `ActualizarObstaculos()`, `HayColision()`, `RevisarColisiones()` y `DibujarJuego()`.
- Un `Main` muy corto que solo llama a `ActualizarJuego()` y `DibujarJuego()` en cada cuadro.

```
struct Bloque
{
    public float x, y, vx, vy;
    public int ancho, alto;
    public Color color;
}

static Bloque jugador;
static Bloque[] obstaculos = new Bloque[NUM_OBSTACULOS];

static void ActualizarObstaculos()
{
    for (int i = 0; i < NUM_OBSTACULOS; i++)
    {
        obstaculos[i].x += obstaculos[i].vx;
        if (obstaculos[i].x + obstaculos[i].ancho < 0)
            obstaculos[i].x += DISTANCIA_REAPARICION;
    }
}

static bool HayColision(Bloque a, Bloque b)
{
    return a.x < b.x + b.ancho && a.x + a.ancho > b.x &&
        a.y < b.y + b.alto && a.y + a.alto > b.y;
}
```

Ejemplo 5. Fragmento de la versión procedural: datos en un struct y funciones separadas.

Un detalle importante: C# obliga a que `Main` esté dentro de una clase, así que la versión procedural conserva `static class Program`. Esa clase no tiene objetos ni herencia; solo funciona como contenedor de funciones, igual que un archivo `.c`.

7. Diferencias entre la versión orientada a objetos y la procedural

Aspecto	Orientada a objetos (nuestra versión)	Procedural (versión con IA)
Unidad básica	Clases que unen datos y métodos	Struct con datos y funciones separadas
Movimiento	Cada clase sobrescribe Update()	Una función por tipo: ActualizarJugador(), ActualizarObstaculos()
Colisiones	jugador.CollisionWith(o) usando Raylib	Función HayColision(a, b) escrita a mano
Estado	Dentro de cada objeto, más el puntaje en Main	Variables globales visibles para todas las funciones
Colección	List<Objeto> polimórfica	Arreglo de tamaño fijo Bloque[3]
Agregar un tipo nuevo	Crear una subclase; el ciclo no cambia	Agregar datos y modificar varias funciones
Constantes	Números dentro de los métodos (-15f, 0.8f)	Constantes con nombre (GRAVEDAD, FUERZA_SALTO)
Lectura	Hay que saltar entre clases para seguir el flujo	Se lee de arriba hacia abajo en orden

Tabla 4. Comparación entre las dos versiones del juego.

Comentarios

Extensibilidad. La mayor ventaja de la versión orientada a objetos aparece cuando el juego crece. Si quisiéramos agregar picos, plataformas o portales, en la versión OO cada uno sería una clase con su propio Update() y Draw(), y el ciclo principal seguiría igual. En la versión procedural habría que agregar campos al struct o nuevos arreglos, y después modificar las funciones de actualización, colisión y dibujo para que los consideren.

Simplicidad. Para un juego tan pequeño, la versión procedural es más fácil de leer: todo está en un solo lugar y el orden de ejecución es evidente. La versión OO reparte la lógica entre varias clases y hay que conocer la herencia para entender qué método se ejecuta.

Manejo del estado. En la versión procedural cualquier función puede modificar al jugador o al puntaje porque son globales. Eso es cómodo, pero en un programa grande hace difícil encontrar quién cambió un dato. En la versión OO cada objeto controla su propia posición y velocidad, lo que se conoce como encapsulamiento.

Buenas prácticas que aprendimos de la versión procedural. La IA reemplazó los números mágicos por constantes con nombre y separó el dibujo de la pantalla de derrota en su propia función. Son mejoras que también podríamos aplicar a nuestra

versión orientada a objetos.

Rendimiento. En este juego no hay diferencia perceptible: los dos corren a 60 cuadros por segundo con tres obstáculos. La elección entre paradigmas aquí depende de la organización del código, no de la velocidad.

8. Conclusiones

Investigar la historia de los paradigmas me ayudó a entender que no se reemplazan entre sí, sino que se acumulan. El paradigma imperativo de los años cincuenta sigue presente en cada ciclo `while` de nuestro juego, la programación estructurada de 1968 explica por qué no usamos `goto`, y las clases de Simula 67 son la misma idea que Caja y Jugador.

Al comparar las dos versiones del juego vi que el mismo comportamiento se puede escribir de formas muy distintas. La versión procedural es más directa para un programa pequeño, pero la orientada a objetos fue la que nos permitió extender el código del profesor sin reescribirlo: solo agregamos subclases y sobrescribimos `Update()`. Esa facilidad para crecer es la razón por la que la mayoría de los lenguajes más usados hoy, como C#, Java y Python, soportan el paradigma orientado a objetos y lo combinan con otros.

Por último, usar inteligencia artificial para generar la versión procedural fue útil, pero tuve que revisar que el resultado se comportara igual que el original. La IA sirve como apoyo para explorar otra forma de programar, siempre que uno entienda el código que entrega.

9. Repositorio del código

El código de las dos versiones, el programa base de clase, la línea del tiempo interactiva y el archivo README con las instrucciones para ejecutar el juego están en GitHub:

<https://github.com/l25210812-creator/geometry-dash-paradigmas>

Referencias

- Dahl, O.-J. y Nygaard, K. (1966). SIMULA: an ALGOL-based simulation language. *Communications of the ACM*, 9(9), 671-678.
- Dijkstra, E. W. (1968). Go To Statement Considered Harmful. *Communications of the ACM*, 11(3), 147-148.
- Microsoft. (2026). *Documentación de C#*. <https://learn.microsoft.com/es-es/dotnet/csharp/>
- Raylib-cs. (2026). *C# bindings for raylib*. <https://github.com/raylib-cs/raylib-cs>
- Santamaria, R. (2026). *raylib: a simple and easy-to-use library to enjoy videogames programming*. <https://www.raylib.com/>

Sebesta, R. W. (2019). *Concepts of Programming Languages* (12.^a ed.). Pearson.

TIOBE Software. (2026). *TIOBE Index for September 2026*. <https://www.tiobe.com/tiobe-index/>

Van Roy, P. (2009). Programming Paradigms for Dummies: What Every Programmer Should Know.
En *New Computational Paradigms for Computer Music*. IRCAM/Delatour.